==========

APEX RETAIL GROUP | ANALYTICS TRAINING PROGRAM
DOCUMENT: CLASS NOTES (SESSION 4)
TOPIC: TABULAR DATA DISCOVERY — READING & INSPECTING REAL DATASETS
LEVEL: BEGINNER-INTERMEDIATE (DAY 4)

==========

1. CONTINUITY & BUSINESS SCENARIO RECAP

In Session 1, you learned how to process numbers using Python variables and lists.
In Session 2, you automated rule-checking using loops, functions, and try/except logic.
In Session 3, you performed vectorized math across multi-dimensional arrays using NumPy.

Now, in your fourth week at Apex Retail Group:

Samantha Reed, E-Commerce Analytics Manager, needs your help with a new project:

> **"Our web store logs thousands of customer transactions weekly into structured CSV files. Before we can build automated revenue models or run promotional campaigns, we need to inspect our weekly raw exports.**
>
> **We don't know if the columns are properly typed, whether dates are parsed correctly, what time range the log actually covers, or if summary statistics reveal structural anomalies in our sales numbers.**
>
> **Can you load this weekly transaction export into Pandas, inspect its metadata and structure, verify column integrity, and produce an initial dataset health and summary report?"**

In this session, you transition from raw NumPy arrays to Pandas—the industry-standard library for tabular data analysis. You will learn how to load external data, inspect DataFrames, evaluate metadata, and establish data health before conducting analysis.

2. FROM NUMPY ARRAYS TO PANDAS DATAFRAMES

While NumPy handles pure numeric grids efficiently, real-world business data consists of named columns containing mixed data types (dates, customer names, category labels, currency numbers, and Booleans).

Pandas introduces two primary data structures built on top of NumPy:
- Pandas Series: A single 1-dimensional labeled column of data.
- Pandas DataFrame: A 2-dimensional tabular data structure with row labels (index) and column headers.

Feature	Numpy Array vs Pandas DataFrame
Column Headers	Numeric indices only vs Named columns
Mixed Data Types	Single data type vs Mixed types per column
File I/O (csv, Excel)	Manual parsing vs Built-in `read_csv()`

3. INSPECTING TABULAR DATA STRUCTURES

A. Loading External CSV Data

Pandas makes reading tabular files straightforward:

```
import pandas as pd
df = pd.read_csv("weekly_transactions.csv")
```

B. Structural Inspection Methods

Before answering business questions, an analyst must inspect dataset geometry:

- `df.head(n)`: Views the first `n` rows (defaults to 5) to inspect structure.
- `df.tail(n)`: Views the last `n` rows to check file ending integrity.
- `df.shape`: Returns a tuple `(rows, columns)` showing dataset dimensions.
- `df.columns`: Lists all column header names.

C. Metadata & Data Type Auditing (`df.info()`)

`df.info()` is the most critical inspection command in Pandas. It reveals:

- Total row count and index range.
- Column names and non-null (non-missing) entry counts.
- Data types (`dtypes`): `int64`, `float64`, `object` (text/strings), `bool`, `datetime64`.

D. Summary Statistics (`df.describe()`)

- `df.describe()` calculates summary statistics (`count`, `mean`, `std`, `min`, `25%`, `50%`, `75%`, `max`) for all numerical columns.
- `df.describe(include='object')` provides frequency summaries for categorical text columns.

4. EXECUTING DATASET DISCOVERY IN PANDAS

Below is the clean, annotated Python code required for Samantha's dataset discovery task:

```python

import pandas as pd

# Step 1: Load external transaction dataset into a Pandas DataFrame
# In a real environment, this reads directly from disk:
# df = pd.read_csv("transactions.csv")
raw_data = {
    "transaction_id": ["TXN-1001", "TXN-1002", "TXN-1003", "TXN-1004",
"TXN-1005", "TXN-1006"],
    "order_date": ["2026-10-01", "2026-10-01", "2026-10-02", "2026-10-02",
"2026-10-03", "2026-10-03"],
    "customer_tier": ["Gold", "Silver", "Bronze", "Gold", "Bronze",
"Silver"],
    "product_category": ["Electronics", "Apparel", "Home", "Electronics",
"Apparel", "Electronics"],
    "item_units": [2, 1, 5, 1, 12, 3],
    "unit_price": [299.99, 45.00, 12.50, 499.99, 15.00, 199.99],
    "order_total": [599.98, 45.00, 62.50, 499.99, 180.00, 599.97]
}

df = pd.DataFrame(raw_data)

# Step 2: Inspect dataset geometry and first few rows
print("--- DATASET SHAPE (Rows, Columns) ---")
```

```
print(df.shape)  # Output: (6, 7)

print("\n--- FIRST 3 TRANSACTIONS ---")
print(df.head(3))

# Step 3: Audit DataFrame Metadata
print("\n--- METADATA & DATA TYPES ---")
df.info()

# Step 4: Generate Summary Statistics
print("\n--- NUMERICAL SUMMARY STATISTICS ---")
print(df.describe())

# Step 5: Select specific columns for targeted inspection
print("\n--- TARGETED COLUMN INSPECTION ---")
print(df[["transaction_id", "product_category", "order_total"]])
```

INTERPRETING NUMBERS & FORMULATING BUSINESS RECOMMENDATIONS

A. Observation (The Raw Data Metadata):

- Dataset Volume: 6 transactions across 7 attributes.
- Column Types Detected:
  - `item_units`: integer (`int64`)
  - `unit_price` & `order_total`: numeric float (`float64`)
  - `order_date`: stored as plain text (`object`) rather than datetime format.
- Summary Statistics Insights:
  - Average Order Total: $331.24 (Min: $45.00 | Max: $599.98).
  - Order Unit Range: 1 unit minimum to 12 units maximum.
  - High Standard Deviation in Order Total ($264.83), reflecting diverse basket sizes.

B. Interpretation & Business Meaning:

- The transaction export loaded cleanly without structural truncation.
- `order_date` is currently parsed as a text string (`object`). While acceptable for basic inspection, time-based trends cannot be calculated until dates are converted to explicit datetime objects in future cleaning steps.
- Large spread between average order ($331.24) and max order ($599.98) indicates distinct customer purchasing behaviors across product categories.

C. Professional Recommendation to Samantha Reed (E-Commerce Manager):

"The weekly transaction dataset export was successfully loaded and audited. The dataset contains 6 records with complete non-null entries across all 7 attributes, averaging $331.24 per order.

Recommendation: Approve the dataset for preliminary analysis. Note that date fields are currently formatted as raw text strings (`object`); I recommend standardizing date formatting before running time-series aggregations."

CORE MINDSET HABITS REINFORCED TODAY

1. Inspect before analyzing: Never run calculations on a dataset without first checking `df.shape`, `df.info()`, and `df.head()`.
2. Verify column data types: A number stored as text (`object`) will break mathematical models downstream.